

Presentation Script

When Should Language Models Search?

M.Tech Thesis Defense | 4 minutes

Md Tanzeel Adam Khan (2411AI67)

[SLIDE 1 — Title]

Good morning everyone. I'm Tanzeel, and this is my M.Tech thesis: **When Should Language Models Search?**

The short answer: we can decide before the model even starts reasoning — and this pre-decision outperforms methods that watch the full reasoning and judge afterward.

[SLIDE 2 — Problem & Approach]

Here's the core tension. On the left: Chain-of-Thought — one straight path, fast, but if it goes wrong early there's no recovery. On the right: A* search — explores branches, finds better paths, but costs 24 times more.

Quick example. “Is shrimp scampi free of plastic?” CoT takes five reasonable-sounding steps and says Yes. All wrong. A* finds “shrimp contain microplastics” in step one and correctly says No.

So the question is: can we predict which method to use from the problem text alone, before generating anything? That's what our pipeline on the right does. We extract 13 features, train an XGBoost router, and it decides CoT or A* with zero LLM cost.

[SLIDE 3 — A* Search: Why & How]

Now, why A* specifically?

We chose A* because it cleanly separates the three ingredients of search: a priority frontier, a path cost, and a heuristic. Tree-of-Thought entangles the search with evaluator quality. MCTS introduces rollout noise. A* lets us isolate when branching itself is responsible for gains — which is exactly our research question.

The formulation: $f = g + h$. The cost g penalizes length and low confidence. The heuristic h estimates remaining effort using depth and entity coverage.

The key property: our heuristic is **admissible**. We prove $h(n) \leq h^*(n)$ for all non-goal nodes. The condition is $\omega_d + \omega_c \leq c_{\min}$. Our values: $0.3 + 0.3 = 0.6$, which is safely below $c_{\min} = 1.01$. This guarantees A* never overestimates and finds optimal paths under budget.

[SLIDE 4 — Topology Features]

Now, how does the router decide?

Look at the table at the top. High branching coefficient, many hops, multiple entities — send to A*. Short, linear, single-step — keep with CoT.

These decisions come from 13 features, all computed from the input text. The starred ones — connectives, hop indicators, context sentences, branching complexity — carry the most weight.

The derived Branching Coefficient — context sentences times hop count divided by question length — alone predicts A* advantage with R-squared 0.94. No LLM call needed. Just text analysis.

[SLIDE 5 — Complementarity & Routing]

Why does routing matter? Because CoT and A* solve genuinely different problems.

Look at the numbers. On StrategyQA, 14% of instances are solved only by CoT, 11% only by A*. On HotpotQA it's 17% each way. On CodeContests, 19% are A*-exclusive. The oracle gap — what perfect routing would achieve — is up to 14 percentage points above the best single method.

How does our router leverage this? It identifies branching problems and sends them to A*, keeps linear ones on CoT. Result: 40% of the oracle gap recovered. And because we only invoke A* on about 30% of instances, the effective cost is 8x CoT, not 24x. Best of both worlds.

[SLIDE 6 — Results]

Validation: 8 benchmarks, 3 model scales, covering QA, math, and code generation.

The color pattern tells the whole story. Red cells: A* wins. Blue: CoT wins. Neither dominates. A* gives plus-19 points on GSM8K at 14B, plus-18 on CodeContests at 0.5B. But CoT wins on HumanEval, MBPP, and LogicQA. The split is systematic, not random — and that's exactly what topology features capture.

[SLIDE 7 — Router Comparison]

The punchline.

This chart shows gap recovery for every routing method we tested. Look at the pattern: CoT alone, A* alone, semantic entropy, LLM-as-Judge, random forest — all negative or barely positive. They're worse than just picking the best fixed method.

Our topology router: plus 40 percent. The only one with substantial positive recovery. On CodeContests: 95% accuracy, 87% gap recovered.

Why does it beat methods that read full reasoning traces? The Fluency-Correctness Asymmetry: 38% of the time, the more fluent trace is actually wrong. Judges get fooled by good writing. Our router doesn't look at writing at all. It routes by structure.

Route by structure, not by trace quality.

[SLIDE 8 — References + Thank You]

These are the key references our work builds upon. Thank you.